Чисельні методи

Лабораторна робота №1

Розв’язання нелінійних рівнянь

Виконав:

Студент КН-24

Бевзюк Н.С.

2026

Варіант **1**

1. Мета роботи.

Здобути практичні навички розроблення алгоритмів і програм чисельних методів розв'язання нелінійних рівнянь f(x) = 0.

2. Завдання.

1) Відділити корені рівняння x - sin x = 0,25.

2) У випадку існування кількох коренів необхідно здійснити уточнення одного з них із заданою точністю ε = 0,001, застосувавши метод половинного ділення та метод хорд.

3) Підрахувати кількість виконаних ітерацій.

4) Побудувати графіки зміни значення x та f(x) за ітераціями.

5) Порівняти результати, зробити перевірку.

3. Розрахункові формули чисельного методу.

Чисельні методи розв'язання нелінійних рівнянь застосовуються для знаходження коренів рівняння вигляду $f(x) = 0$у випадках, коли аналітичний розв'язок відсутній або його отримання є складним. Нехай функція $f(x)$ є неперервною на відрізку $[a, b]$і виконується умова $f(a) \cdot f(b) < 0$, що гарантує існування принаймні одного кореня на цьому інтервалі.

У роботі використано два класичні методи — метод половинного ділення (дихотомії) та метод хорд.

Метод половинного ділення ґрунтується на послідовному звуженні відрізка, який містить корінь. На кожній ітерації обчислюється середина відрізка $c_k = \frac{a_k+b_k}{2}$, після чого визначається підвідрізок, на якому функція змінює знак: якщо $f(a_k) \cdot f(c_k) < 0$, то новий інтервал має вигляд $[a_k, c_k]$, інакше $[c_k, b_k]$. Процес повторюється до виконання умови зупинки, яка у практичній реалізації може мати вигляд $|\boldsymbol{f(c_k)}| < \boldsymbol{\varepsilon}$, де ε— задана точність.

Метод хорд базується на апроксимації функції прямою лінією, проведеною через точки $(a_k, f(a_k))$ та $(b_k, f(b_k))$. Рівняння цієї прямої дозволяє визначити точку її перетину з віссю абсцис, яка обчислюється за формулою $c_k = a_k - \frac{f(a_k)}{f(b_k)-f(a_k)}(b_k - a_k)$. Отримана точка використовується для звуження інтервалу аналогічно до методу дихотомії з урахуванням зміни знаку функції. Умовою завершення ітераційного процесу зазвичай є критерій $|\boldsymbol{c_k - c_{k-1}}| < \boldsymbol{\varepsilon}$. Обидва

методи дозволяють знайти наближене значення кореня з наперед заданою похибкою та широко застосовуються у чисельному аналізі для розв'язання прикладних задач.

4. Текст програми.

Посилання на GitHub програми - https://github.com/Bevziuk2005/numerical-methods

ПОЧАТОК КОДУ

```
import tkinter as tk
from tkinter import ttk
import math
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import ticker
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
NavigationToolbar2Tk

# 1. Метод половинного ділення
def bisection_method(func, a, b, eps):
    iteration = 0  # лічильник
    x_values = []      # список наближень x
    fx_values = []     # список значень f(x)

    while True:
        iteration += 1

        c = (a + b) / 2
        # Збереження значень у списки
        x_values.append(c)
        fx_values.append(func(c))

        # перевірка умови виходу
        if abs(func(c)) < eps:
            break

        # зміна відрізку через перевірку зміни знаку
        if func(a) * func(c) < 0:
            b = c   #корінь в [a, c]
        else:
            a = c   #корінь в [c, b]
```

```
    # відповідь
    root = (a + b) / 2
    return root, iteration, x_values, fx_values

# 2. Метод хорд
def chord_method(func, a, b, eps):
    iteration = 0  # лічильник
    x_values = []      # список наближень x
    fx_values = []     # список значень f(x)
    c_prev = None   # попереднє c, для умови виходу

    while True:
        iteration += 1

        c = a - (func(a) / (func(b) - func(a))) * (b - a)
        x_values.append(c)
        fx_values.append(func(c))

        # умова виходу від 2+ ітерації
        if c_prev is not None:
            if abs(c - c_prev) < eps:
                break

        if func(a) * func(c) < 0:
            b = c
        else:
            a = c
        c_prev = c

    root = c
    return root, iteration, x_values, fx_values

# прибирає все зайве при роботі з графіком збіжності
class MinimalToolbar(NavigationToolbar2Tk):
    toolitems = (
        ('Pan', 'Pan axes with left mouse, zoom with right', 'move', 'pan'),
        ('Zoom', 'Zoom to rectangle', 'zoom_to_rect', 'zoom'),
    )

# функція для округлення відповіді до похибки ε
def round_by_eps(value, eps):
    if eps <= 0:
        return value
```

```
    digits = max(0, int(math.ceil(-math.log10(eps))))
    return round(value, digits)

# відмальовка GUI
class RootApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Чисельні методи знаходження кореня")

        tk.Label(root, text="Введіть функцію f(x):").pack()
        self.func_entry = tk.Entry(root, width=50)
        self.func_entry.insert(0, "x - math.sin(x) - 0.25")
        self.func_entry.pack()

        tk.Button(root, text="Створити графік",
command=self.plot_function).pack(pady=5)

        self.params_frame = tk.Frame(root)
        self.params_frame.pack(pady=5)

        tk.Label(self.params_frame, text="a:").grid(row=0, column=0)
        self.a_entry = tk.Entry(self.params_frame, width=8)
        self.a_entry.grid(row=0, column=1)
        tk.Label(self.params_frame, text="b:").grid(row=0, column=2)
        self.b_entry = tk.Entry(self.params_frame, width=8)
        self.b_entry.grid(row=0, column=3)
        tk.Label(self.params_frame, text="eps:").grid(row=0, column=4)
        self.eps_entry = tk.Entry(self.params_frame, width=8)
        self.eps_entry.grid(row=0, column=5)

        tk.Button(root, text="Підрахувати", command=self.calculate).pack(pady=5)
        self.result_frame = tk.Frame(root)
        self.result_frame.pack()

    # відмалювання графіку функції
    def plot_function(self):
        expr = self.func_entry.get()

        def f(x):
            return eval(expr)

        x = np.linspace(-100, 100, 5000)
```

```
    y = [f(i) for i in x]

    window = tk.Toplevel(self.root)
    window.title("Графік функції")

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(x, y)
    ax.axhline(0)
    ax.set_title("Графік функції")
    ax.set_xlabel("x")
    ax.set_ylabel("f(x)")
    ax.grid(True)

    canvas = FigureCanvasTkAgg(fig, master=window)
    canvas.draw()
    canvas.get_tk_widget().pack()

    toolbar = MinimalToolbar(canvas, window)
    toolbar.update()
    canvas.get_tk_widget().pack()

# робота та виведення методів
def calculate(self):
    expr = self.func_entry.get()
    a = float(self.a_entry.get())
    b = float(self.b_entry.get())
    eps = float(self.eps_entry.get())

    def f(x):
        return eval(expr)

    root_bis, iter_bis, x_bis, fx_bis = bisection_method(f, a, b, eps)
    root_ch, iter_ch, x_ch, fx_ch = chord_method(f, a, b, eps)
    root_bis_r = round_by_eps(root_bis, eps)
    root_ch_r = round_by_eps(root_ch, eps)

    # контрольне виведення через print
    print("\nМетод половинного ділення")
    print("Ітерацій:", iter_bis)
    print("Корінь:", root_bis_r)
    print("\nМетод хорд")
    print("Ітерацій:", iter_ch)
    print("Корінь:", root_ch_r)
```

```
    # таблиця
    for widget in self.result_frame.winfo_children():
        widget.destroy()

    columns = ("Метод", "Корінь", "Ітерації")
    tree = ttk.Treeview(self.result_frame, columns=columns, show="headings")

    for col in columns:
        tree.heading(col, text=col)

    tree.insert("", "end", values=("Половинне ділення", root_bis_r, iter_bis))
    tree.insert("", "end", values=("Метод хорд", root_ch_r, iter_ch))

    tree.pack()

    tk.Button(self.result_frame, text="Графіки збіжності",
              command=lambda: self.plot_convergence(x_bis, x_ch)).pack(pady=5)

# відмалювання графіку збіжності
def plot_convergence(self, x_bis, x_ch):
    window = tk.Toplevel(self.root)
    window.title("Графік збіжності")

    max_len = max(len(x_bis), len(x_ch))

    # Мінімальна ширина з автозбільшенням
    base_width = 8
    dynamic_width = max(base_width, max_len * 0.4)

    fig, ax = plt.subplots(figsize=(dynamic_width, 5))

    iterations_bis = list(range(1, len(x_bis) + 1))
    iterations_ch = list(range(1, len(x_ch) + 1))

    ax.plot(iterations_bis, x_bis, marker='o')
    ax.plot(iterations_ch, x_ch, marker='x')

    ax.set_title("Збіжність методів (наближення x)")
    ax.set_xlabel("Ітерація")
    ax.set_ylabel("x")

    ax.set_xticks(range(1, max_len + 1))
```

```
        ax.xaxis.set_major_locator(ticker.MultipleLocator(1))
        ax.grid(True)

        canvas = FigureCanvasTkAgg(fig, master=window)
        canvas.draw()
        canvas.get_tk_widget().pack()

        # таблиця з даними
        table = ttk.Treeview(window, columns=("№", "x_bis", "x_ch"),
show="headings")

        table.heading("№", text="№")
        table.heading("x_bis", text="x (половинне)")
        table.heading("x_ch", text="x (хорди)")

        for i in range(max_len):
            xb = x_bis[i] if i < len(x_bis) else ""
            xc = x_ch[i] if i < len(x_ch) else ""
            table.insert("", "end", values=(i + 1, xb, xc))

        table.pack(fill="both", expand=True)

        window.minsize(900, 600)

# запуск GUI
if __name__ == "__main__":
    root = tk.Tk()
    app = RootApp(root)
    root.mainloop()
```

КІНЕЦЬ КОДУ

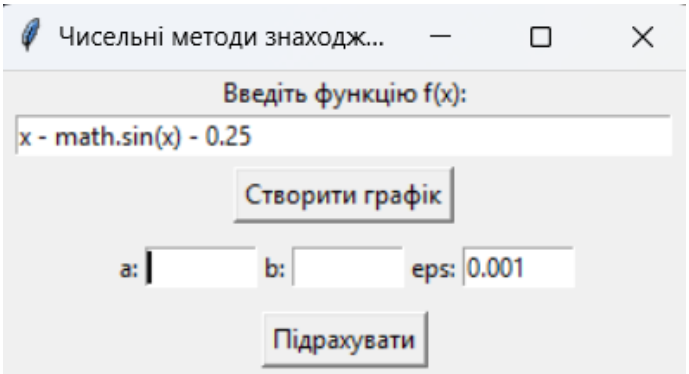
5. Результат роботи програми та графік зміни значення x за ітераціями.

Дані:

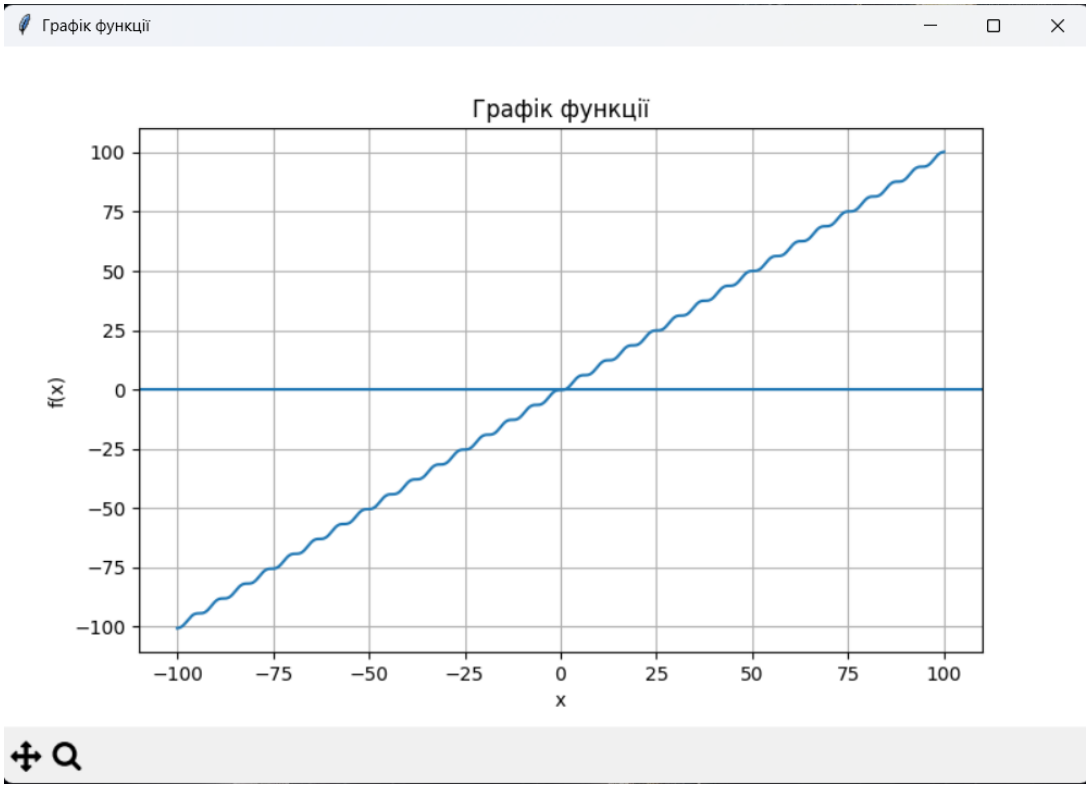
Рівняння - x - sin x = 0,25

Оцінка похибки ε - 0,001

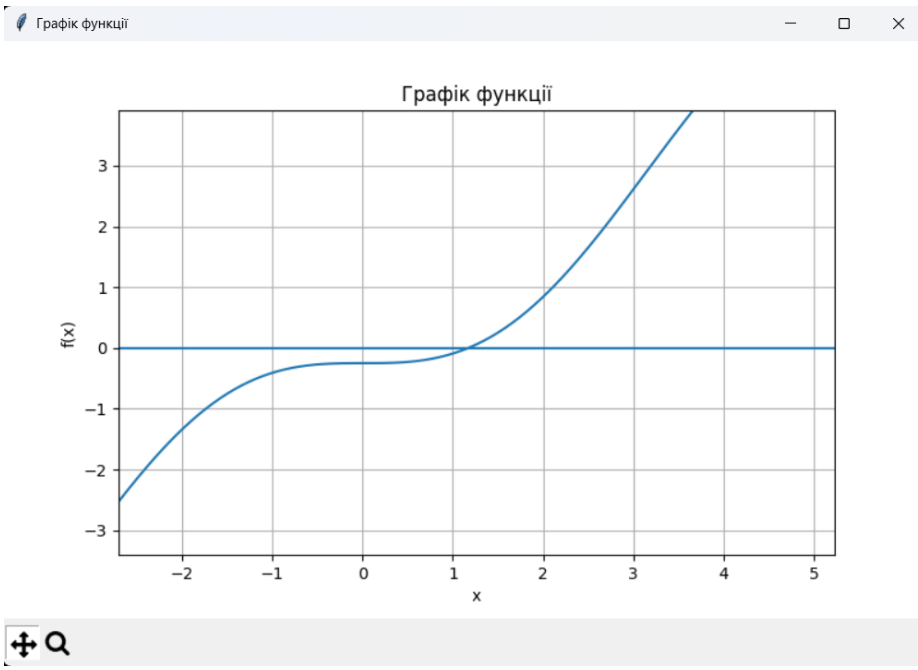
Запуск програми:



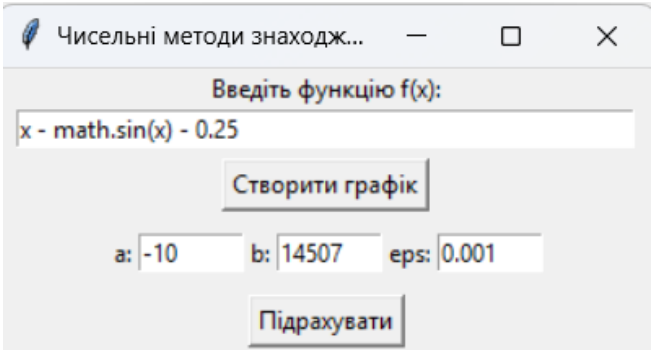
Натискання на кнопку “Створити графік”:



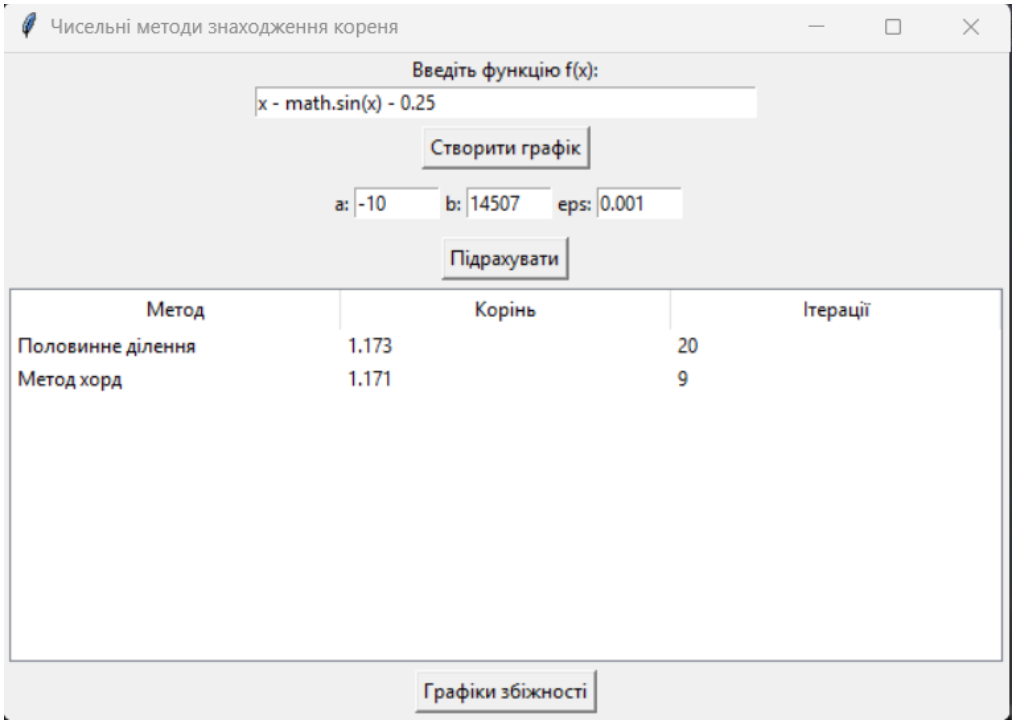
Наближення до місця знаходження корення для визначення a та b:



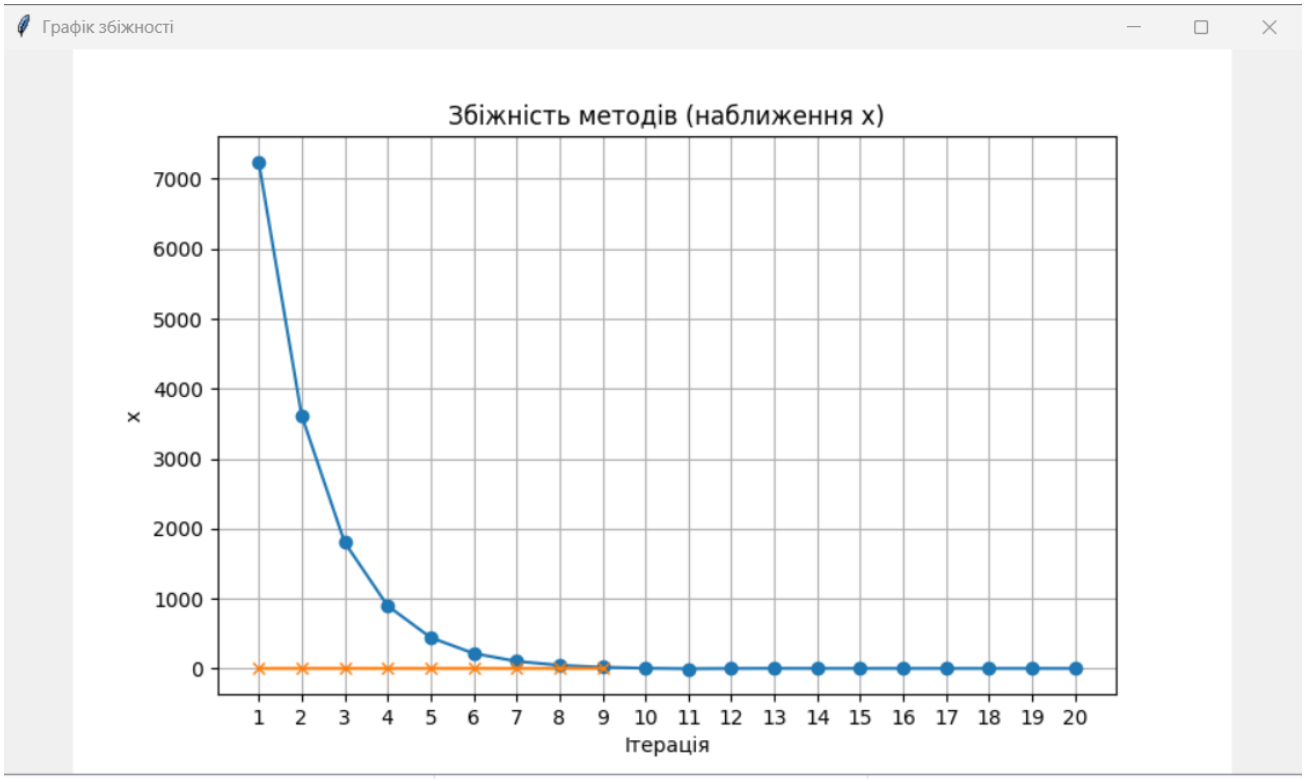
Визначення a та b і внесення даних:



Натискання на кнопку “Підрахувати”:

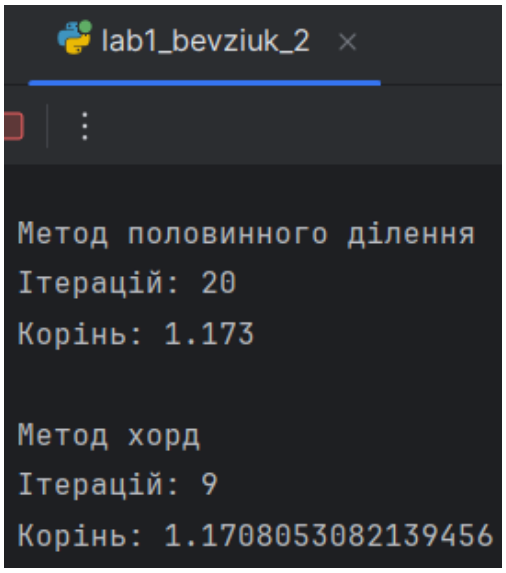


Натискання на кнопку “Графіки збіжності”:



№	x (половинне)	x (хорди)
1	7248.5	0.7930460535483004
2	3619.25	0.9624766384393324
3	1804.625	1.0705976194176618
4	897.3125	1.1274808287052833
5	443.65625	1.153331514073721
6	216.828125	1.164118503552848
7	103.4140625	1.1684395077664944
8	46.70703125	1.1701404503466522
9	18.353515625	1.1708053082139456
10	4.1767578125	
11	-2.91162109375	
12	0.632568359375	
13	2.4046630859375	

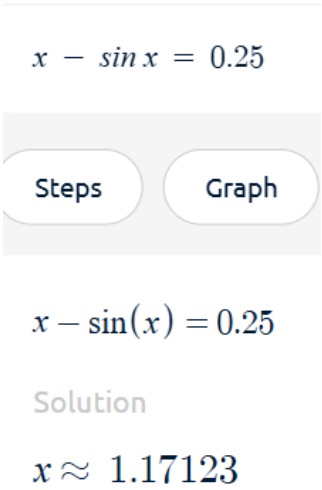
Виведення в консолі:



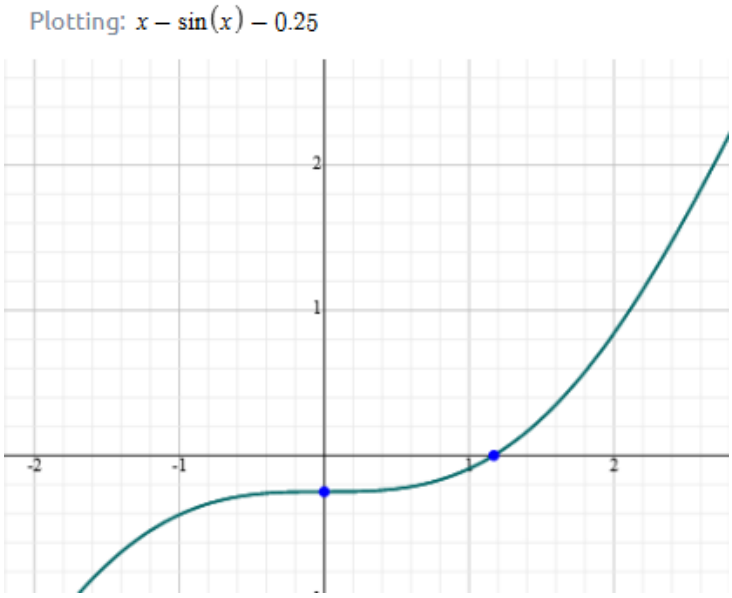
6. Висновки.

У даній роботі було реалізовано чисельне розв'язання нелінійного рівняння$x - \sin(x) = 0.25$ із застосуванням двох класичних методів чисельного аналізу — методу половинного ділення та методу хорд.

Перед проведенням порівняльного аналізу було здійснено перевірку правильності отриманих результатів за допомогою онлайн-сервісу Symbolab - https://www.symbolab.com/solver?or=gms&query=x+-+sin+x+%3D+0.25 , який визначає корінь рівняння як: $x \approx 1{,}17123$



і графік як



Отримані в програмі значення кореня знаходяться в межах заданої похибки $\varepsilon = 0.001$, що підтверджує коректність реалізованих алгоритмів та правильність математичних обчислень.

Побудований графік функції дозволяє наочно переконатися у наявності єдиного кореня на заданому інтервалі та візуально проаналізувати поведінку функції поблизу точки перетину з віссю абсцис.

Порівняння методів показало, що метод половинного ділення потребує **більшої** кількості ітерацій, натомість метод хорд продемонстрував **швидшу** збіжність і потребував вдвічі менше ітерацій для досягнення тієї ж точності.

Графіки збіжності підтверджують теоретичні властивості методів: послідовність наближень методу хорд швидше стабілізується поблизу кореня, тоді як метод половинного ділення поступово звужує інтервал пошуку.

Отже, реалізовані методи коректно розв’язують поставлену задачу, а результати узгоджуються з незалежним математичним ресурсом. Метод половинного ділення є більш надійним з точки зору гарантованої збіжності, тоді як метод хорд є ефективнішим за швидкістю досягнення необхідної точності.